

```
#VENKATESH GAURI SHANKAR
```

```
##Exp 7: To execute data normalization and segmentation methods and their different variants.
```

```
# importing packages
```

```
import pandas as pd
```

```
# create data
```

```
df = pd.DataFrame([
    [180000, 110, 18.9, 1400],
    [360000, 905, 23.4, 1800],
    [230000, 230, 14.0, 1300],
    [60000, 450, 13.5, 1500]],

    columns=['Col A', 'Col B',
             'Col C', 'Col D'])
```

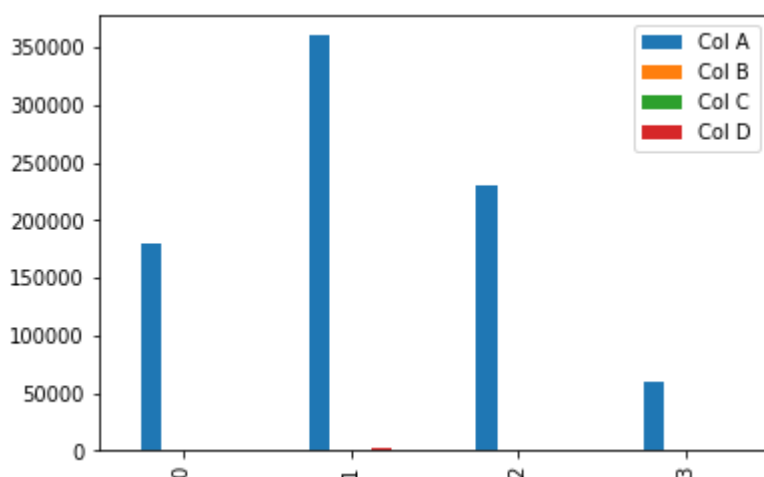
```
# view data
```

```
display(df)
```

	Col A	Col B	Col C	Col D
0	180000	110	18.9	1400
1	360000	905	23.4	1800
2	230000	230	14.0	1300
3	60000	450	13.5	1500

```
import matplotlib.pyplot as plt
df.plot(kind = 'bar')
```

<matplotlib.axes._subplots.AxesSubplot at 0x7f35171cb190>



```
#Using The maximum absolute scaling- maximum absolute value/decimal scaling
```

```
# copy the data
df_max_scaled = df.copy()

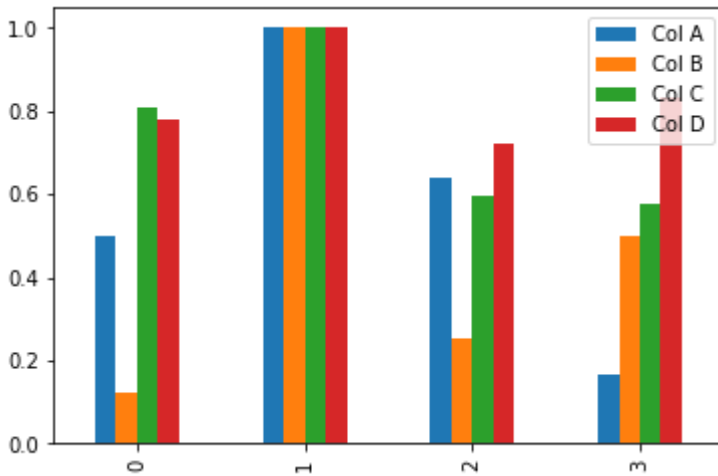
# apply normalization techniques
for column in df_max_scaled.columns:
    df_max_scaled[column] = df_max_scaled[column] / df_max_scaled[column].abs().max()

# view normalized data
display(df_max_scaled)
```

	Col A	Col B	Col C	Col D
0	0.500000	0.121547	0.807692	0.777778
1	1.000000	1.000000	1.000000	1.000000
2	0.638889	0.254144	0.598291	0.722222
3	0.166667	0.497238	0.576923	0.833333

```
import matplotlib.pyplot as plt
df_max_scaled.plot(kind = 'bar')
```

<matplotlib.axes._subplots.AxesSubplot at 0x7f35171bda50>



##Using The min-max feature scaling-subtracting the minimum value of the feature then dividing by th

```
# copy the data
df_min_max_scaled = df.copy()

# apply normalization techniques
for column in df_min_max_scaled.columns:
    df_min_max_scaled[column] = (df_min_max_scaled[column] - df_min_max_scaled[column].min()) / (df_

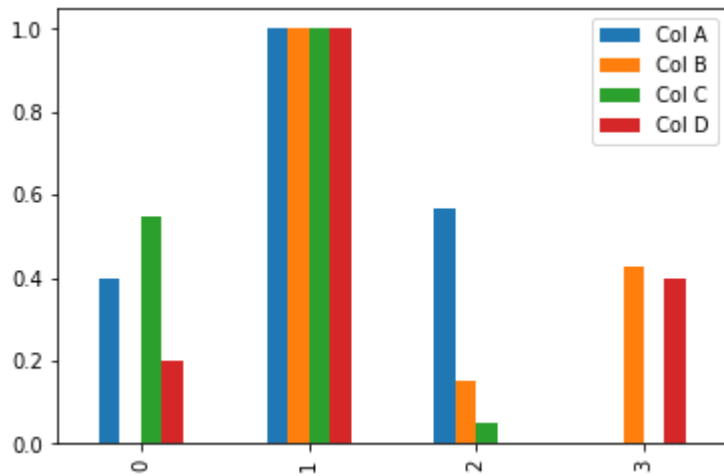
# view normalized data
print(df_min_max_scaled)
```

	Col A	Col B	Col C	Col D
0	0.400000	0.000000	0.545455	0.2
1	1.000000	1.000000	1.000000	1.0

```
2 0.566667 0.150943 0.050505 0.0
3 0.000000 0.427673 0.000000 0.4
```

```
import matplotlib.pyplot as plt
df_min_max_scaled.plot(kind = 'bar')
```

<matplotlib.axes._subplots.AxesSubplot at 0x7f35160d6fd0>



##Using The z-score method-distribution with a mean of 0 and a typical deviation of 1

```
# copy the data
df_z_scaled = df.copy()

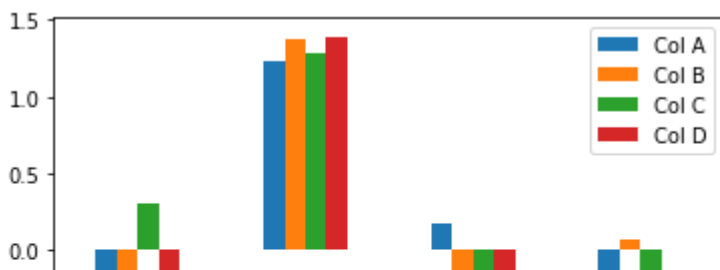
# apply normalization techniques
for column in df_z_scaled.columns:
    df_z_scaled[column] = (df_z_scaled[column] -
                           df_z_scaled[column].mean()) / df_z_scaled[column].std()

# view normalized data
display(df_z_scaled)
```

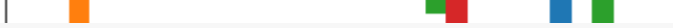
	Col A	Col B	Col C	Col D
0	-0.221422	-0.895492	0.311486	-0.46291
1	1.227884	1.373564	1.278167	1.38873
2	0.181163	-0.552993	-0.741122	-0.92582
3	-1.187625	0.074922	-0.848531	0.00000

```
import matplotlib.pyplot as plt
df_z_scaled.plot(kind='bar')
```

<matplotlib.axes._subplots.AxesSubplot at 0x7f3515fda710>



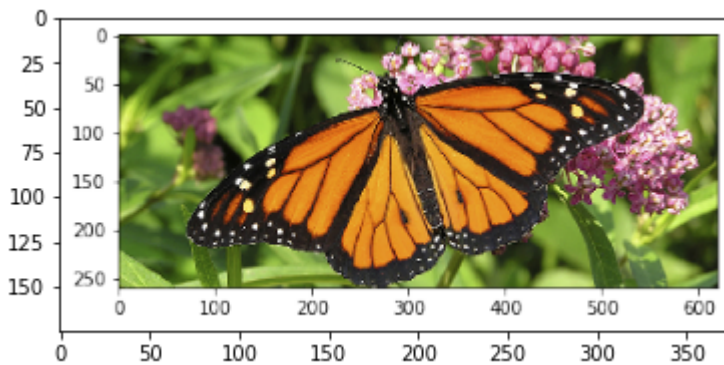
Segmentation



##Laplacian Operator for segmentation

```
import matplotlib.pyplot as plt
%matplotlib inline
Laplacian=plt.imread("download.png")
plt.imshow(Laplacian)
```

<matplotlib.image.AxesImage at 0x7f3515f23d50>



```
import cv2
import matplotlib.pyplot as plt

# Open the image
img = cv2.imread('download.png')

# Apply gray scale
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply gaussian blur
blur_img = cv2.GaussianBlur(gray_img, (3, 3), 0)

# Positive Laplacian Operator
laplacian = cv2.Laplacian(blur_img, cv2.CV_64F)

plt.figure()
plt.title('Laplacian')
plt.imsave('Laplacian.png', laplacian, cmap='gray', format='png')
plt.imshow(laplacian, cmap='gray')
plt.show()
```



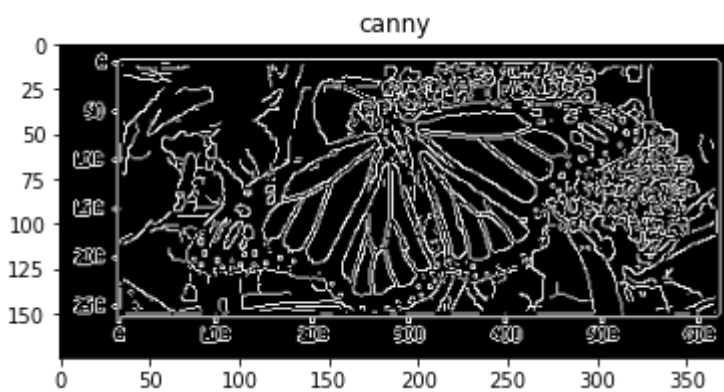
```
##Canny Operator for segmentation
```

```
import cv2
import matplotlib.pyplot as plt

# Open the image
img = cv2.imread('download.png')

# Apply Canny
edges = cv2.Canny(img, 100, 200, 3, L2gradient=True)

plt.figure()
plt.title('canny')
plt.imsave('canny.png', edges, cmap='gray', format='png')
plt.imshow(edges, cmap='gray')
plt.show()
```



```
#Gray scale brain image
```

```
##Laplacian Operator for segmentation- B&W
```

```
import matplotlib.pyplot as plt
%matplotlib inline
Laplacian=plt.imread("brain.png")
plt.imshow(Laplacian)
```

<matplotlib.image.AxesImage at 0x7f350b48f350>

0

```
import cv2
import matplotlib.pyplot as plt

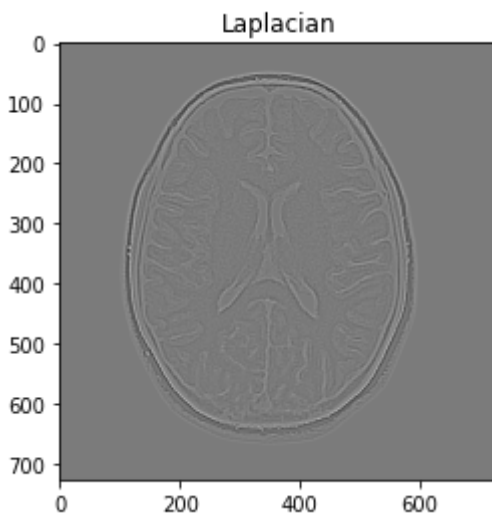
# Open the image
img = cv2.imread('brain.png')

# Apply gray scale
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply gaussian blur
blur_img = cv2.GaussianBlur(gray_img, (3, 3), 0)

# Positive Laplacian Operator
laplacian = cv2.Laplacian(blur_img, cv2.CV_64F)

plt.figure()
plt.title('Laplacian')
plt.imsave('Laplacian.png', laplacian, cmap='gray', format='png')
plt.imshow(laplacian, cmap='gray')
plt.show()
```



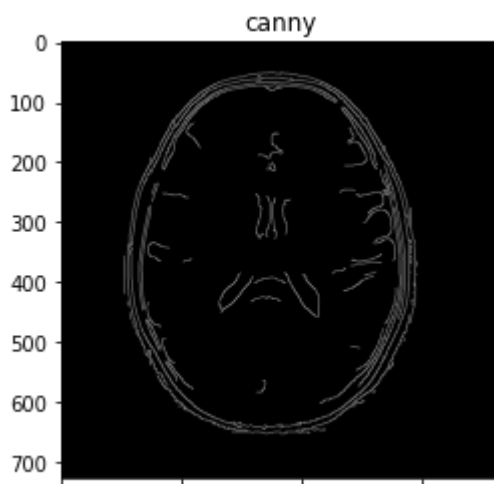
##Canny Operator for segmentation-B&W

```
import cv2
import matplotlib.pyplot as plt

# Open the image
img = cv2.imread('brain.png')

# Apply Canny
edges = cv2.Canny(img, 100, 200, 3, L2gradient=True)

plt.figure()
plt.title('canny')
plt.imsave('canny.png', edges, cmap='gray', format='png')
plt.imshow(edges, cmap='gray')
plt.show()
```



#Salt and Pepper Noise segmentation

```
import matplotlib.pyplot as plt
%matplotlib inline
img=plt.imread("mr_bean.jpeg")
plt.imshow(img)
```

<matplotlib.image.AxesImage at 0x7f350aa4e450>



```
import random
import cv2

def add_noise(img):

    # Getting the dimensions of the image
    row , col = img.shape

    # Randomly pick some pixels in the
    # image for coloring them white
    # Pick a random number between 300 and 10000
    number_of_pixels = random.randint(300, 10000)
    for i in range(number_of_pixels):

        # Pick a random y coordinate
        y_coord=random.randint(0, row - 1)

        # Pick a random x coordinate
        x_coord=random.randint(0, col - 1)

        # Color that pixel to white
```

```
img[y_coord][x_coord] = 255
```

```
# Randomly pick some pixels in  
# the image for coloring them black  
# Pick a random number between 300 and 10000  
number_of_pixels = random.randint(300 , 10000)  
for i in range(number_of_pixels):
```

```
    # Pick a random y coordinate  
    y_coord=random.randint(0, row - 1)
```

```
    # Pick a random x coordinate  
    x_coord=random.randint(0, col - 1)
```

```
    # Color that pixel to black  
    img[y_coord][x_coord] = 0
```

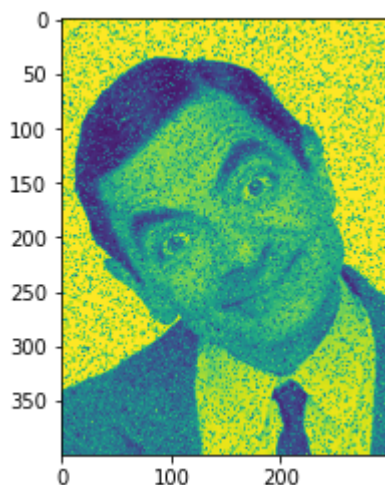
```
return img
```

```
# salt-and-pepper noise can  
# be applied only to grayscale images  
# Reading the color image in grayscale image  
img = cv2.imread('mr_bean.jpeg',  
                 cv2.IMREAD_GRAYSCALE)
```

```
#Storing the image  
cv2.imwrite('salt-and-pepper-bean.jpeg',  
           add_noise(img))
```

True

```
plt.imshow(img)  
plt.show()
```



Colab paid products - [Cancel contracts here](#)

